# Unit IV

## 4. Managing Data Storage, Advanced Components of Android and Location & Map

**4.1 Shared Preferences**
**4.2 Internal Storage, External Storage**
**4.3 SQLite Databases**
**4.4 Content provider and Remote Databases**
**4.5 Web App, JSON Parsing**
**4.6 Google maps, Using GPS to find current location**
**4.7 Sensors, bluetooth/Wi-Fi Connectivity**

---

## 4.1 Shared Preferences:

Here's a complete explanation on **Shared Preferences** in Android:

**Shared Preferences** is a lightweight **key-value storage** system used to save **small amounts of data** (like settings or user preferences).

Data is stored in an **XML file** and persists across app sessions.

**Use Cases:**

Saving login status (e.g., remember me).
Storing app settings like theme, language.
Saving user choices like last opened tab, volume level, etc.

**Basic Operations:**

**1. Creating and Saving Data:**

**java**

```
SharedPreferences sharedPref = getSharedPreferences("MyPrefs",
MODE_PRIVATE);
SharedPreferences.Editor editor = sharedPref.edit();
editor.putString("username", "John");
editor.putBoolean("isLoggedIn", true);
editor.apply(); // or commit()
```

**2. Retrieving Data:**

**java**

```
SharedPreferences sharedPref = getSharedPreferences("MyPrefs",
MODE_PRIVATE);
String user = sharedPref.getString("username", "defaultUser");
boolean isLoggedIn = sharedPref.getBoolean("isLoggedIn", false);
```

**3. Removing Data:**

**java**

```
editor.remove("username").apply();
```

**4. Clearing All Preferences:**

**java**

```
editor.clear().apply();
```

**Advantages:**

Simple to use.
No database required.
Quick read/write for small data.

## 4.2 Internal Storage, External Storage:

Here's a clear and concise explanation on **Internal Storage** and **External Storage** in Android:

**1. Internal Storage:**

Data is saved in the **private storage area of the app**.
Files stored here **cannot be accessed by other apps**.
Suitable for storing sensitive or app-specific data like user settings or cached content.

**Saving File:**

**java**

```
String data = "Hello Android";
FileOutputStream fos = openFileOutput("myfile.txt", MODE_PRIVATE);
fos.write(data.getBytes());
fos.close();
```

**Reading File:**

**java**

```
FileInputStream fis = openFileInput("myfile.txt");
```

**2. External Storage:**

Refers to **shared storage**, like the phone's SD card or emulated storage.
Used to store **large, non-sensitive files**, such as photos, videos, or downloads.
Requires **runtime permissions** in newer Android versions.

**Checking & Writing:**

**java**

```
if(Environment.MEDIA_MOUNTED.equals(Environment.getExternalStorageState()))
{
    File file = new File(getExternalFilesDir(null), "data.txt");
    FileOutputStream fos = new FileOutputStream(file);
    fos.write("Hello".getBytes());
    fos.close();
}
```

**Permissions:**

**In `AndroidManifest.xml` (for external storage):**

**xml**

```
<uses-permission
    android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission
    android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

**Key Differences:**

| Feature | Internal Storage | External Storage |
|---|---|---|
| **Accessibility** | App-only | Shared with other apps |
| **Security** | More secure | Less secure |
| **Size Limit** | Limited to app space | Depends on available device space |
| **Permissions** | No runtime permission needed | Requires runtime permission |

## 4.3 SQLite Databases:

Here's a clear and concise explanation on **SQLite Databases** in Android:

**SQLite Databases in Android:**

**SQLite** is a **lightweight, built-in relational database** used in Android to store structured data. It is ideal for storing **persistent, structured** data like user info, app data, etc.

**Key Features:**

Stores data in **tables with rows and columns**.
No need for a server – database runs locally in the app.
SQL commands like `SELECT`, `INSERT`, `UPDATE`, `DELETE` are used.

**Creating and Using SQLite:**

**1. Create a Database Helper Class:**

**java**

```
public class MyDatabaseHelper extends SQLiteOpenHelper {
    public MyDatabaseHelper(Context context) {
        super(context, "MyDB", null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE Users(id INTEGER
      PRIMARY KEY, name TEXT)");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int
  newVersion) {
        db.execSQL("DROP TABLE IF EXISTS Users");
        onCreate(db);
    }
}
```

**2. Insert Data:**

**java**

```
SQLiteDatabase db = dbHelper.getWritableDatabase();
ContentValues values = new ContentValues();
```

```
values.put("name", "John");
db.insert("Users", null, values);
```

**3. Query Data:**

**java**

```
Cursor cursor = db.query("Users", null, null, null, null, null, null);
while (cursor.moveToNext()) {
    String name =
        cursor.getString(cursor.getColumnIndex("name"));
}
cursor.close();
```

**Advantages:**

Easy to implement.
No extra setup required.
Efficient for small to medium datasets.

## 4.4 Content provider and Remote Databases:

Here's a complete and concise explanation on **Content Provider** and **Remote Databases** in Android:

**1. Content Provider:**

A **Content Provider** manages access to a **structured set of data**.
It enables **data sharing** between different apps.
Works with **URI-based requests** (like `content://contacts/people`).
Used for accessing shared data such as contacts, media, or calendar entries.

**Key Methods:**

`query()`, `insert()`, `update()`, `delete()`, `getType()`

**Example Use:**

**java**

```
Cursor cursor = getContentResolver().query(
    ContactsContract.Contacts.CONTENT_URI,
    null, null, null, null);
```

**Use Cases:**

Access system data (e.g., contacts, messages).
Share your app's data with other apps securely.

**2. Remote Databases:**

A **Remote Database** stores data on a **server** (not on the device).
Used for **cloud storage**, user syncing, and **multi-device access**.
Requires **internet connection** and APIs (like REST or GraphQL) to communicate.

**Common Technologies:**

Firebase Realtime Database / Firestore
MySQL, MongoDB (used with custom backends)
REST APIs (via Retrofit, Volley, etc.)

**Example with Firebase:**

**java**

```
DatabaseReference ref =FirebaseDatabase.getInstance().getReference("users");
ref.child("123").setValue("John");
```

**Key Differences:**

| Feature | Content Provider | Remote Database |
|---|---|---|
| **Location** | Local (on device) | Server-based (cloud) |
| **Purpose** | Data sharing across apps | Data syncing across devices/users |
| **Requires Internet** | No | Yes |
| **Usage** | Contacts, Media, etc. | User profiles, chat, online backups |

## 4.5 Web App, JSON Parsing:

Here's a concise and clear explanation on **Web App** and **JSON Parsing** in Android:

**1. Web App in Android:**

A **Web App** is an application that runs inside a **WebView** component in Android and loads **web content** (HTML, CSS, JavaScript) from the internet or local assets.

Useful for displaying web pages or hybrid apps (combination of native + web).

**WebView Example:**

**java**

```
WebView webView = findViewById(R.id.webView);
webView.getSettings().setJavaScriptEnabled(true);
webView.loadUrl("https://www.example.com");
```

**Use Cases:**

Show websites inside an app.
Load online forms, dashboards, or documentation.
Display content without rebuilding the app.

**2. JSON Parsing in Android:**

**JSON (JavaScript Object Notation)** is a lightweight format used for data exchange between Android apps and servers (APIs).

Parsing means **converting JSON data into Java objects**.

**Sample JSON:**

**json**

```
{
    "name": "Alice",
    "age": 25
}
```

**Parsing Example (using JSONObject):**

**java**

```
String jsonString = "{\"name\":\"Alice\", \"age\":25}";
JSONObject jsonObject = new JSONObject(jsonString);
String name = jsonObject.getString("name");
int age = jsonObject.getInt("age");
```

**Other Libraries for JSON Parsing:**

**Gson** (by Google)
**Moshi**
**Jackson**

**Summary:**

| Concept | Web App | JSON Parsing |
|---|---|---|
| **Purpose** | Load/display web content in app | Read data from APIs (servers) |
| **Key Class** | `WebView` | `JSONObject`, `JSONArray`, `Gson` |
| **Internet Needed** | Yes (usually) | Yes (for online APIs) |
| **Use Case** | Hybrid apps, websites | API response handling, data display |

## 4.6 Google maps, Using GPS to find current location:

Here's a complete and clear explanation on **Google Maps** and **Using GPS to Find Current Location** in Android:

**1. Google Maps in Android:**

Google Maps is integrated in Android apps using the **Google Maps API**.
It allows displaying maps, markers, routes, and places inside your app.

**Steps to Use:**

1. Add **Google Maps SDK** in `build.gradle`.
2. Get a **Google Maps API key** from the Google Cloud Console.
3. Use a `MapFragment` or `MapView` in layout.

**Example Code:**

**java**

```
GoogleMap googleMap;

@Override
public void onMapReady(GoogleMap map) {
    googleMap = map;
    LatLng location = new LatLng(37.7749, -122.4194);
    googleMap.addMarker(new
        MarkerOptions().position(location).title("San Francisco"));
    googleMap.moveCamera
        (CameraUpdateFactory.newLatLngZoom(location, 10));
}
```

**2. Using GPS to Find Current Location:**

Android uses the **FusedLocationProviderClient** to get the **current GPS location** efficiently.
It combines **GPS, Wi-Fi, and cell tower** data for accuracy and battery saving.

**Example Code:**

**java**

```
FusedLocationProviderClient fusedLocationClient =
        LocationServices.getFusedLocationProviderClient(this);

fusedLocationClient.getLastLocation()
.addOnSuccessListener(this, location -> {
        if (location != null) {
                double latitude = location.getLatitude();
                double longitude = location.getLongitude();
                // Use coordinates
        }
});
```

**Permissions Needed in `AndroidManifest.xml`:**

**xml**

```
<uses-permission
        android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission
        android:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

**Don't forget to request runtime permissions** in your activity for Android 6.0+.

**Summary Table:**

| Feature | Google Maps | GPS Location |
|---|---|---|
| **Purpose** | Show maps & markers | Get device's real-time location |
| **API Used** | Google Maps API | FusedLocationProviderClient |
| **Internet Needed** | Yes (for map tiles) | Not always (but recommended) |
| **Use Case** | Navigation, showing places | Tracking user, showing current place |

## 4.7 Sensors, bluetooth/Wi-Fi Connectivity:

Here's a complete explanation on **Sensors** and **Bluetooth/Wi-Fi Connectivity** in Android:

**1. Sensors in Android:**

Android devices include various **sensors** that provide data about device orientation, motion, environment, etc.

Managed using the **SensorManager** class.

**Common Sensors:**

**Accelerometer** – Detects movement/shaking.
**Gyroscope** – Measures rotation.
**Proximity sensor** – Detects nearby objects.
**Light sensor** – Measures ambient light.

**Basic Usage:**

**java**

```java
SensorManager sensorManager = (SensorManager)
    getSystemService(Context.SENSOR_SERVICE);
Sensor accelerometer =
    sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);

SensorEventListener listener = new SensorEventListener() {
    @Override
    public void onSensorChanged(SensorEvent event) {
        float x = event.values[0]; // X-axis data
    }

    @Override
    public void onAccuracyChanged(Sensor sensor, int accuracy) {}
};

sensorManager.registerListener(listener, accelerometer,
    SensorManager.SENSOR_DELAY_NORMAL);
```

**2. Bluetooth/Wi-Fi Connectivity:**

**Bluetooth:**

Used for **wireless short-range communication** between devices.
Requires **Bluetooth permission** and adapter setup.

**java**

```java
BluetoothAdapter bluetoothAdapter = BluetoothAdapter.getDefaultAdapter();
if (!bluetoothAdapter.isEnabled()) {
    Intent enableBtIntent = new
        Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE);
    startActivityForResult(enableBtIntent, REQUEST_ENABLE_BT);
}
```

**Wi-Fi:**

Allows the app to connect to the **internet** or **local networks** using Wi-Fi.

**java**

```java
WifiManager wifiManager = (WifiManager)
    getApplicationContext().getSystemService(Context.WIFI_SERVICE);
wifiManager.setWifiEnabled(true); // Turns on Wi-Fi
```

**Permissions in Manifest:**

**xml**

```xml
<uses-permission android:name="android.permission.BLUETOOTH"/>
<uses-permission
    android:name="android.permission.BLUETOOTH_ADMIN"/>
<uses-permission
    android:name="android.permission.ACCESS_WIFI_STATE"/>
```

```
<uses-permission
        android:name="android.permission.CHANGE_WIFI_STATE"/>
```

**Summary Table:**

| Feature | Sensors | Bluetooth/Wi-Fi |
|---|---|---|
| **Use** | Detect device motion, light, etc. | Device communication, internet |
| **API Used** | SensorManager | BluetoothAdapter / WiFiManager |
| **Permissions** | No (for most sensors) | Yes |
| **Example Use Case** | Fitness apps, games | File transfer, chat, IoT, networking |